



</> HackWithInfy 2026

Topic-wise practice bank

☰ 13/13 topics with full solutions

1. Arrays ✨

2. Strings ✨

3. Sliding Window ✨

4. Binary Search ✨

5. Hashing ✨

6

1. Arrays

7 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

▼ Rearrange Array Elements by Sign



Easy

Problem: Array of size n (n even) with equal positive and negative numbers. Rearrange so positives and negatives alternate, starting with positive, preserving relative order within each sign.

Approach: Use a result array of size n . Fill even indices (0,2,4...) with positives in order, odd indices (1,3,5...) with negatives in order. Single pass, $O(n)$ extra space.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def rearrange_array(nums):
    n = len(nums)
    result = [0] * n
    pos_idx, neg_idx = 0, 1
    for num in nums:
        if num >= 0:
            result[pos_idx] = num
            pos_idx += 2
        else:
            result[neg_idx] = num
            neg_idx += 2
    return result
```

Complexity: Time $O(n)$, Space $O(n)$ — required to preserve order.

Edge cases: All positives/negatives violates the equal-count precondition; array of size 0 or 2; confirm whether 0 counts as positive.

▼ Maximum Product Subarray



Medium

Problem: Find the contiguous subarray with the largest product.

Approach: Track running max AND running min ending at each index, since a negative number can flip min into max. Candidates each step: num , $\text{maxSoFar} * \text{num}$, $\text{minSoFar} * \text{num}$.

[Java](#)[Python](#)[Solve on LeetCode](#)

PYTHON

```
def max_product(nums):
    max_so_far = min_so_far = result = nums[0]
    for num in nums[1:]:
        if num < 0:
            max_so_far, min_so_far = min_so_far, max_so_far
        max_so_far = max(num, max_so_far * num)
        min_so_far = min(num, min_so_far * num)
        result = max(result, max_so_far)
    return result
```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: Array contains 0 (resets candidates); single element; all-negative with odd count.

Rotate Matrix In-Place by 90°



Medium

Problem: $N \times N$ matrix, rotate 90° clockwise using $O(1)$ extra space.

Approach: Transpose the matrix (swap $[i][j]$ with $[j][i]$), then reverse each row.

[Java](#)[Python](#)[Solve on LeetCode](#)

PYTHON

```
def rotate(matrix):
    n = len(matrix)
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    for row in matrix:
        row.reverse()
```

Complexity: Time $O(n^2)$, Space $O(1)$.

Edge cases: 1×1 matrix; non-square input is out of scope for this approach; CCW rotation needs reverse-then-transpose instead.

Merge Overlapping Intervals



Medium

Problem: Given $[start, end]$ pairs, merge all overlaps.

Approach: Sort by start time. Iterate, merging into the last interval in the result if $current.start \leq last.end$.

[Java](#)[Python](#)[Solve on LeetCode](#)

PYTHON

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    result = []
```

```

for start, end in intervals:
    if not result or result[-1][1] < start:
        result.append([start, end])
    else:
        result[-1][1] = max(result[-1][1], end)
return result

```

Complexity: Time $O(n \log n)$, Space $O(n)$.

Edge cases: Empty input; single interval; intervals that touch exactly $([1,4],[4,5])$; fully nested intervals.

Trapping Rain Water



Hard

Problem: Given elevation heights, compute total trapped water.

Approach: Two-pointer, $O(1)$ space. Water at $i = \min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$. Always move the side with the smaller max.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def trap(height):
    left, right = 0, len(height) - 1
    left_max = right_max = water = 0
    while left < right:
        if height[left] < height[right]:
            left_max = max(left_max, height[left])
            water += left_max - height[left]
            left += 1
        else:
            right_max = max(right_max, height[right])
            water += right_max - height[right]
            right -= 1
    return water

```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: Array length < 3 (0 water); all same height; strictly increasing/decreasing.

Median of Two Sorted Arrays



Hard

Problem: Two sorted arrays sizes m, n . Find median in $O(\log(\min(m,n)))$.

Approach: Binary search on the smaller array for a partition point where all left elements $\leq$ all right elements across both arrays.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def find_median_sorted_arrays(nums1, nums2):
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

```

```

m, n = len(nums1), len(nums2)
low, high = 0, m
while low <= high:
    cut1 = (low + high) // 2
    cut2 = (m + n + 1) // 2 - cut1
    left1 = float('-inf') if cut1 == 0 else nums1[cut1 - 1]
    left2 = float('-inf') if cut2 == 0 else nums2[cut2 - 1]
    right1 = float('inf') if cut1 == m else nums1[cut1]
    right2 = float('inf') if cut2 == n else nums2[cut2]
    if left1 <= right2 and left2 <= right1:
        if (m + n) % 2 == 0:
            return (max(left1, left2) + min(right1, right2)) / 2
        return max(left1, left2)
    elif left1 > right2:
        high = cut1 - 1
    else:
        low = cut1 + 1
raise ValueError("Input arrays are not sorted")

```

Complexity: Time $O(\log(\min(m,n)))$, Space $O(1)$.

Edge cases: One array empty; very different sizes (always search the smaller); odd vs even total length changes the formula.

Next Permutation



Hard

Problem: Rearrange to the lexicographically next greater permutation, in-place. If none exists, return the lowest.

Approach: From the right, find pivot i where $\text{nums}[i] < \text{nums}[i+1]$. Find rightmost $j > i$ with $\text{nums}[j] > \text{nums}[i]$, swap, then reverse the suffix after i .

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def next_permutation(nums):
    n = len(nums)
    i = n - 2
    while i >= 0 and nums[i] >= nums[i + 1]:
        i -= 1
    if i >= 0:
        j = n - 1
        while nums[j] <= nums[i]:
            j -= 1
        nums[i], nums[j] = nums[j], nums[i]
    nums[i + 1:] = reversed(nums[i + 1:])

```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: Already fully descending (wraps to ascending); size 1 array; all duplicate elements.

2. Strings

7 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

Group Anagrams



Easy

Problem: Group a list of strings so each group is a set of anagrams.

Approach: Use the sorted string (or a 26-count signature) as a hashmap key.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def group_anagrams(strs):
    groups = {}
    for s in strs:
        key = ''.join(sorted(s))
        groups.setdefault(key, []).append(s)
    return list(groups.values())
```

Complexity: Time $O(n * k \log k)$, Space $O(n * k)$.

Edge cases: Empty strings; non-lowercase/unicode characters; single-element input.

Longest Common Prefix



Easy

Problem: Find the longest common prefix across an array of strings.

Approach: Take the first string as candidate; trim it until it prefixes every string.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def longest_common_prefix(strs):
    if not strs:
        return ""
    prefix = strs[0]
    for s in strs[1:]:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
```

```
        if not prefix:
            return ""
    return prefix
```

Complexity: Time $O(S)$ total chars, Space $O(1)$ extra.

Edge cases: Empty array; single string; no common prefix at all.

▼ Longest Palindromic Substring



Medium

Problem: Find the longest palindromic substring.

Approach: Expand around center for both odd-length and even-length palindromes at every index.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def longest_palindrome(s):
    if not s:
        return ""
    start, end = 0, 0

    def expand(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return right - left - 1

    for i in range(len(s)):
        length = max(expand(i, i), expand(i, i + 1))
        if length > end - start + 1:
            start = i - (length - 1) // 2
            end = i + length // 2
    return s[start:end + 1]
```

Complexity: Time $O(n^2)$, Space $O(1)$. Manacher's gives $O(n)$ as a follow-up.

Edge cases: Empty string; single character; all identical characters; no palindrome > 1 char.

▼ String Compression (In-Place)



Medium

Problem: "aaabbc" $\rightarrow$ "a3b2c1", modify array in place, $O(1)$ space.

Approach: Two pointers: read scans runs, write writes char + count back into same array.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def compress(chars):
    write = read = 0
    n = len(chars)
    while read < n:
        current_char = chars[read]
```

```

count = 0
while read < n and chars[read] == current_char:
    read += 1
    count += 1
chars[write] = current_char
write += 1
if count > 1:
    for digit in str(count):
        chars[write] = digit
        write += 1
return write

```

Complexity: Time $O(n)$, Space $O(1)$ extra.

Edge cases: Single char; all same char; no repeats at all; counts ≥ 10 need multi-digit writes.

Minimum Window Substring



Medium

Problem: Smallest substring of S containing all characters of T (with multiplicity).

Approach: Sliding window + frequency map of T. Expand right until valid, contract left while still valid, track minimum.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

from collections import Counter

def min_window(s, t):
    if not s or not t:
        return ""
    need = Counter(t)
    required = len(need)
    window_counts = {}
    formed = 0
    left = 0
    min_len, min_start = float('inf'), 0
    for right, char in enumerate(s):
        window_counts[char] = window_counts.get(char, 0) + 1
        if char in need and window_counts[char] == need[char]:
            formed += 1
        while left <= right and formed == required:
            if right - left + 1 < min_len:
                min_len, min_start = right - left + 1, left
            lc = s[left]
            window_counts[lc] -= 1
            if lc in need and window_counts[lc] < need[lc]:
                formed -= 1
            left += 1
    return "" if min_len == float('inf') else s[min_start:min_start + min_len]

```

Complexity: Time $O(|S| + |T|)$, Space $O(|S| + |T|)$.

Edge cases: T longer than S; T has repeated chars (must match multiplicity); no valid window exists.

Regular Expression Matching (. and *)



Hard

Problem: Implement regex matching supporting '.' and '*'.

Approach: DP table $dp[i][j]$. For '*' consider zero occurrences (skip pair) and one more occurrence (if char matches).

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def is_match(s, p):
    m, n = len(s), len(p)
    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = True
    for j in range(1, n + 1):
        if p[j - 1] == '*':
            dp[0][j] = dp[0][j - 2]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            sc, pc = s[i - 1], p[j - 1]
            if pc == '.' or pc == sc:
                dp[i][j] = dp[i - 1][j - 1]
            elif pc == '*':
                dp[i][j] = dp[i][j - 2]
                prev_p = p[j - 2]
                if prev_p == '.' or prev_p == sc:
                    dp[i][j] = dp[i][j] or dp[i - 1][j]
```



HackWithInfy 2026

Student Browser



Demo Student

STUDENT

[Logout](#)

Complexity: Time $O(m*n)$, Space $O(m*n)$, reducible to $O(n)$.

Edge cases: Pattern like a^* matching empty string; consecutive * patterns; empty pattern vs non-empty string.

Word Break II



Hard

Problem: Return ALL ways to segment a string into dictionary words.

Approach: Backtracking with memoization on start index — try every valid dictionary prefix, recurse on remainder, combine.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def word_break(s, word_dict):
    dict_set = set(word_dict)
    memo = {}

    def backtrack(start):
```



```
    if start in memo:
        return memo[start]
    if start == len(s):
        return [""]
    result = []
    for end in range(start + 1, len(s) + 1):
        word = s[start:end]
        if word in dict_set:
            for sub in backtrack(end):
                result.append(word + (" " if not sub else " " + sub))
    memo[start] = result
    return result

return backtrack(0)
```

Complexity: Time $O(n^2)$ distinct states plus output-sensitive combination; Space $O(n^2)$.

Edge cases: No valid segmentation (empty list); whole string is one word; exponential-output adversarial cases need a reachability pre-check.



</> HackWithInfy 2026

Topic-wise practice bank

☰ 13/13 topics with full solutions

1. Arrays ✨

2. Strings ✨

3. Sliding Window ✨

4. Binary Search ✨

5. Hashing ✨

6

3. Sliding Window

6 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

Maximum Average Subarray I (Max Sum Subarray)



Easy

Problem: Fixed-size window, find max sum or average.

Approach: Sum first window, then slide: subtract outgoing element, add incoming element.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def max_sum_subarray(nums, k):
    window_sum = sum(nums[:k])
    max_sum = window_sum
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        max_sum = max(max_sum, window_sum)
    return max_sum
```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: $k >$ array length (invalid); $k ==$ array length; negative numbers present.

Longest Substring Without Repeating Characters



Medium

Problem: Find length of the longest substring with all unique characters.

Approach: Sliding window + hashmap of last-seen index. On a repeat inside the window, jump left past the previous occurrence.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def length_of_longest_substring(s):
    last_seen = {}
    left = max_len = 0
    for right, char in enumerate(s):
        if char in last_seen and last_seen[char] >= left:
            left = last_seen[char] + 1
```

```
        left = last_seen[char] + 1
        last_seen[char] = right
        max_len = max(max_len, right - left + 1)
    return max_len
```

Complexity: Time $O(n)$, Space $O(\min(n, \text{charset}))$.

Edge cases: Empty string; all unique; all identical characters; the $\geq$ left check is essential.

✓ Longest Substring with At Most K Distinct Characters



Medium

Problem: Sliding window with a frequency map; shrink from left while distinct count $> k$.

Approach: Expand right always; while freq map size $> k$, shrink from left.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def length_of_longest_substring_k_distinct(s, k):
    if k == 0:
        return 0
    freq = {}
    left = max_len = 0
    for right, char in enumerate(s):
        freq[char] = freq.get(char, 0) + 1
        while len(freq) > k:
            lc = s[left]
            freq[lc] -= 1
            if freq[lc] == 0:
                del freq[lc]
            left += 1
        max_len = max(max_len, right - left + 1)
    return max_len
```

Complexity: Time $O(n)$, Space $O(k)$.

Edge cases: $k = 0$; $k \geq$ distinct chars in s ; empty string.

✓ Fruit Into Baskets



Medium

Problem: Same template as K-distinct with $k=2$ hardcoded — recognize the disguise.

Approach: Reuse the exact K-distinct sliding window template with $k=2$.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def total_fruit(fruits):
    freq = {}
    left = max_len = 0
    for right, fruit in enumerate(fruits):
        freq[fruit] = freq.get(fruit, 0) + 1
        while len(freq) > 2:
```

```

        lf = fruits[left]
        freq[lf] -= 1
        if freq[lf] == 0:
            del freq[lf]
        left += 1
    max_len = max(max_len, right - left + 1)
    return max_len

```

Complexity: Time $O(n)$, Space $O(1)$ (at most 3 keys).

Edge cases: Fewer than 2 distinct fruit types total; empty array.

Sliding Window Maximum



Hard

Problem: Max in every window of size K, return array of maxes.

Approach: Monotonic decreasing deque of indices. Front is always current window max. Pop back while smaller than new element; pop front if out of window.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

from collections import deque

def max_sliding_window(nums, k):
    dq = deque()
    result = []
    for i, num in enumerate(nums):
        while dq and dq[0] < i - k + 1:
            dq.popleft()
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result

```

Complexity: Time $O(n)$ amortized, Space $O(k)$.

Edge cases: $k = 1$; $k = \text{array length}$; duplicate values; empty array.

Subarrays with K Different Integers



Hard

Problem: Count subarrays with exactly K distinct integers.

Approach: $\text{exactly}(K) = \text{atMost}(K) - \text{atMost}(K-1)$, each implemented as a shrinking window.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def subarrays_with_k_distinct(nums, k):
    def at_most_k_distinct(k):
        if k < 0:

```

```
        return 0
    freq = {}
    left = count = 0
    for right, num in enumerate(nums):
        freq[num] = freq.get(num, 0) + 1
        while len(freq) > k:
            ln = nums[left]
            freq[ln] -= 1
            if freq[ln] == 0:
                del freq[ln]
            left += 1
        count += right - left + 1
    return count

return at_most_k_distinct(k) - at_most_k_distinct(k - 1)
```

Complexity: Time $O(n)$, Space $O(k)$.

Edge cases: $k = 0$; $k >$ distinct elements in array; the atMost trick is the key insight, don't track 'exactly K' directly.



</> HackWithInfy 2026

Topic-wise practice bank

☰ 13/13 topics with full solutions

1. Arrays ⚡

2. Strings ⚡

3. Sliding Window ⚡

4. Binary Search ⚡

5. Hashing ⚡

6

4. Binary Search

7 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

▼ Search in Rotated Sorted Array



Easy

Problem: Array sorted ascending then rotated at an unknown pivot. Search target in $O(\log n)$.

Approach: Standard binary search; at each step determine which half is sorted, check if target lies in that half's range.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1
```

Complexity: Time $O(\log n)$, Space $O(1)$.

Edge cases: Not rotated at all; single element; target absent; duplicates break the log-n guarantee.

▼ Find First and Last Position of Element in Sorted Array



Easy

Problem: Return [first, last] index of target, or [-1,-1].

Approach: Two binary searches, one biased to leftmost occurrence, one to rightmost.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def search_range(nums, target):
    def find_bound(find_first):
        left, right, result = 0, len(nums) - 1, -1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                result = mid
                if find_first:
                    right = mid - 1
            else:
                left = mid + 1
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1
        return result

    return [find_bound(True), find_bound(False)]
```

Complexity: Time $O(\log n)$, Space $O(1)$.

Edge cases: Target not present; empty array; target appears once; whole array is the target.

✓ Search a 2D Matrix II



Medium

Problem: Each row sorted L-R, each column sorted top-bottom. Search for target.

Approach: Start top-right. If current > target move left; if current < target move down.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def search_matrix(matrix, target):
    if not matrix or not matrix[0]:
        return False
    row, col = 0, len(matrix[0]) - 1
    while row < len(matrix) and col >= 0:
        if matrix[row][col] == target:
            return True
        elif matrix[row][col] > target:
            col -= 1
        else:
            row += 1
    return False
```

Complexity: Time $O(m + n)$, Space $O(1)$.

Edge cases: Empty matrix/rows; target outside value range; 1×1 matrix.

Find Peak Element



Medium

Problem: Array not necessarily sorted; find any index that is a local peak, in $O(\log n)$.

Approach: Binary search comparing `nums[mid]` to `nums[mid+1]` to decide which half must contain a peak.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def find_peak_element(nums):
    left, right = 0, len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] > nums[mid + 1]:
            right = mid
        else:
            left = mid + 1
    return left
```

Complexity: Time $O(\log n)$, Space $O(1)$.

Edge cases: Single element; strictly increasing/decreasing; multiple peaks (any valid one is fine).

Koko Eating Bananas



Medium

Problem: Find minimum integer eating speed k so all piles finish within H hours.

Approach: Binary search over the ANSWER SPACE (1..max pile), not the array. Check feasibility in $O(n)$ per candidate speed.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
import math

def min_eating_speed(piles, h):
    left, right = 1, max(piles)
    while left < right:
        mid = (left + right) // 2
        hours = sum(math.ceil(pile / mid) for pile in piles)
        if hours <= h:
            right = mid
        else:
            left = mid + 1
    return left
```

Complexity: Time $O(n \log(\max \text{ pile}))$, Space $O(1)$.

Edge cases: H equal to number of piles forces max speed; large values need 64-bit accumulation in Java.

Median of Two Sorted Arrays



Hard

Problem: Same problem as Arrays 1.6 — revisit as a partition-based binary search on which array contributes more elements to the left half.

Approach: See Arrays topic for the full annotated solution; the search space here is a partition count, not array values.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
# See Arrays topic -> "Median of Two Sorted Arrays" for the full solution.
```

Complexity: Time $O(\log(\min(m,n)))$, Space $O(1)$.

Edge cases: Cross-reference only — solved fully under Arrays.

Split Array Largest Sum



Hard

Problem: Split array into m contiguous subarrays to minimize the largest subarray sum.

Approach: Binary search on answer (candidate max sum, range $\max(\text{nums}).. \text{sum}(\text{nums})$). Greedily count subarrays needed; feasible if $\text{count} \leq m$.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def split_array(nums, m):
    left, right = max(nums), sum(nums)

    def can_split(max_sum):
        count, current_sum = 1, 0
        for num in nums:
            if current_sum + num > max_sum:
                count += 1
                current_sum = num
            else:
                current_sum += num
        return count <= m

    while left < right:
        mid = (left + right) // 2
        if can_split(mid):
            right = mid
        else:
            left = mid + 1
    return left
```

Complexity: Time $O(n \log(\text{sum} - \text{max}))$, Space $O(1)$.

Edge cases: $m = 1$ (whole-array sum); $m \geq \text{array length}$ (max single element); one dominant element.



</> HackWithInfy 2026

Topic-wise practice bank

☰ 13/13 topics with full solutions

1. Arrays ✨ 2. Strings ✨ 3. Sliding Window ✨ 4. Binary Search ✨ 5. Hashing ✨ 6

5. Hashing

6 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

▼ Two Sum



Easy

Problem: Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Approach: Use a Hash Map to store complement values (`target - current element`) mapped to their indices for $O(1)$ lookups.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: Negative numbers; exact pair adding to target; duplicate values.

▼ Subarray Sum Equals K



Easy

Problem: Find the total number of contiguous subarrays whose sum equals to `k`.

Approach: Maintain running prefix sum and store frequency of each sum in a hashmap. Count matches where `(currSum - k)` exists.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def subarray_sum(nums, k):
    counts = {0: 1}
```

```

curr_sum = count = 0
for num in nums:
    curr_sum += num
    if (curr_sum - k) in counts:
        count += counts[curr_sum - k]
    counts[curr_sum] = counts.get(curr_sum, 0) + 1
return count

```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: Negative numbers in array; $k = 0$; entire array sums to k .

Longest Consecutive Sequence



Medium

Problem: Find length of the longest consecutive elements sequence in unsorted array in $O(n)$ time.

Approach: Put elements into a HashSet. Only expand sequences starting at num if (num - 1) is NOT present in the set.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def longest_consecutive(nums):
    num_set = set(nums)
    max_len = 0
    for num in num_set:
        if num - 1 not in num_set:
            curr = num
            curr_len = 1
            while curr + 1 in num_set:
                curr += 1
                curr_len += 1
            max_len = max(max_len, curr_len)
    return max_len

```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: Empty array; duplicate numbers; negative values.

Insert Delete GetRandom $O(1)$



Medium

Problem: Design data structure supporting insert, remove, and getRandom in $O(1)$ average time.

Approach: Combine dynamic array with a HashMap (val $\rightarrow$ array index). Swap target element with last element for $O(1)$ removal.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

import random

class RandomizedSet:
    def __init__(self):

```

```

        self.nums = []
        self.val_map = {}

    def insert(self, val: int) -> bool:
        if val in self.val_map: return False
        self.val_map[val] = len(self.nums)
        self.nums.append(val)
        return True

    def remove(self, val: int) -> bool:
        if val not in self.val_map: return False
        idx = self.val_map[val]
        last = self.nums[-1]
        self.nums[idx] = last
        self.val_map[last] = idx
        self.nums.pop()
        del self.val_map[val]
        return True

    def getRandom(self) -> int:
        return random.choice(self.nums)

```

Complexity: Time $O(1)$ average, Space $O(n)$.

Edge cases: Removing element that is already at the end of the array.

Isomorphic Strings



Medium

Problem: Check if characters in *s* can be replaced 1-to-1 to get *t*.

Approach: Store the last seen 1-based index for characters in both strings using fixed 256 size arrays.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def is_isomorphic(s: str, t: str) -> bool:
    m1, m2 = {}, {}
    for c1, c2 in zip(s, t):
        if m1.get(c1) != m2.get(c2):
            return False
        m1[c1] = m2[c2] = c1 + "-" + c2
    return True

```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: Multiple characters trying to map to the same target character.

LFU Cache



Hard

Problem: Design LFU (Least Frequently Used) Cache with get & put in $O(1)$.

Approach: Map key to node, and map frequency to a Doubly Linked List of nodes. Track minFreq.

PYTHON

```
from collections import defaultdict

class Node:
    def __init__(self, k, v):
        self.key, self.val, self.freq = k, v, 1
        self.prev = self.next = None

class DLLList:
    def __init__(self):
        self.head, self.tail = Node(0, 0), Node(0, 0)
        self.head.next = self.tail; self.tail.prev = self.head
        self.size = 0
    def add(self, n):
        n.next = self.head.next; n.prev = self.head
        self.head.next.prev = n; self.head.next = n; self.size += 1
    def remove(self, n):
        n.prev.next = n.next; n.next.prev = n.prev; self.size -= 1
    def pop_tail(self):
        if self.size == 0: return None
        n = self.tail.prev; self.remove(n); return n

class LFUCache:
    def __init__(self, capacity: int):
        self.cap, self.min_freq = capacity, 0
        self.key_map, self.freq_map = {}, defaultdict(DLLList)

    def get(self, key: int) -> int:
        if key not in self.key_map: return -1
        node = self.key_map[key]
        self._update(node)
        return node.val

    def put(self, key: int, value: int) -> None:
        if self.cap == 0: return
        if key in self.key_map:
            node = self.key_map[key]
            node.val = value
            self._update(node)
        else:
            if len(self.key_map) >= self.cap:
                evicted = self.freq_map[self.min_freq].pop_tail()
                del self.key_map[evicted.key]
            node = Node(key, value)
            self.key_map[key] = node
            self.freq_map[1].add(node)
            self.min_freq = 1

    def _update(self, node):
        flist = self.freq_map[node.freq]
        flist.remove(node)
```

```
if node.freq == self.min_freq and flist.size == 0: self.min_freq += 1
node.freq += 1
self.freq_map[node.freq].add(node)
```

Complexity: Time $O(1)$, Space $O(\text{capacity})$.

Edge cases: Capacity 0; tie-breaker for identical frequencies (evicts least recently used).



</> HackWithInfy 2026

Topic-wise practice bank

☰ 13/13 topics with full solutions

1. Arrays ✨ 2. Strings ✨ 3. Sliding Window ✨ 4. Binary Search ✨ 5. Hashing ✨ 6

6. Stack & Queue

7 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

Valid Parentheses



Easy

Problem: Determine if string of brackets '()[]{}' is valid and matched in order.

Approach: Use a Stack. Push expected closing brackets for open brackets; pop and compare for closing brackets.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def is_valid(s: str) -> bool:
    stack = []
    mapping = {"(": ")", "[": "]", "{": "}"
    for char in s:
        if char in mapping:
            top = stack.pop() if stack else '#'
            if mapping[char] != top: return False
        else:
            stack.append(char)
    return not stack
```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: Odd length string; starting with closing bracket; remaining unclosed brackets.

Implement Queue using Stacks



Easy

Problem: Implement FIFO queue using two stacks (inStack and outStack).

Approach: Push to inStack. For pop/peek, transfer elements to outStack if outStack is empty.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
class MyQueue:
    def __init__(self):
```



```

        self.in_stk, self.out_stk = [], []
    def push(self, x: int) -> None:
        self.in_stk.append(x)
    def pop(self) -> int:
        self.peek()
        return self.out_stk.pop()
    def peek(self) -> int:
        if not self.out_stk:
            while self.in_stk:
                self.out_stk.append(self.in_stk.pop())
        return self.out_stk[-1]
    def empty(self) -> bool:
        return not self.in_stk and not self.out_stk

```

Complexity: Time $O(1)$ amortized, Space $O(n)$.

Edge cases: Popping or peeking when queue is empty.

Next Greater Element II



Medium

Problem: Find next greater element in a circular array.

Approach: Monotonic stack holding indices, iterating through array twice (index $i \% n$).

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def next_greater_elements(nums):
    n = len(nums)
    res = [-1] * n
    stack = []
    for i in range(2 * n):
        while stack and nums[stack[-1]] < nums[i % n]:
            res[stack.pop()] = nums[i % n]
        if i < n: stack.append(i)
    return res

```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: Strictly decreasing array; all elements equal.

Min Stack



Medium

Problem: Design stack that supports push, pop, top, and retrieving min in $O(1)$.

Approach: Maintain a primary stack and a minStack that records running minimums.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

class MinStack:
    def __init__(self):
        self.stack, self.min_stack = [], []

```

```

def push(self, val: int) -> None:
    self.stack.append(val)
    if not self.min_stack or val <= self.min_stack[-1]:
        self.min_stack.append(val)
def pop(self) -> None:
    if self.stack.pop() == self.min_stack[-1]:
        self.min_stack.pop()
def top(self) -> int: return self.stack[-1]
def getMin(self) -> int: return self.min_stack[-1]

```

Complexity: Time $O(1)$, Space $O(n)$.

Edge cases: Duplicate minimum values pushed consecutively.

Evaluate Reverse Polish Notation



Medium

Problem: Evaluate arithmetic expression in Reverse Polish Notation (postfix).

Approach: Use a Stack. Push numbers; pop two operands for operators and push result.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def eval_rpn(tokens):
    stack = []
    for t in tokens:
        if t in "+-*/":
            b = stack.pop()
            a = stack.pop()
            if t == '+': stack.append(a + b)
            elif t == '-': stack.append(a - b)
            elif t == '*': stack.append(a * b)
            elif t == '/': stack.append(int(a / b))
        else:
            stack.append(int(t))
    return stack[0]

```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: Integer division truncates toward zero; negative numbers.

Largest Rectangle in Histogram



Hard

Problem: Find area of largest rectangle in histogram bar heights.

Approach: Monotonic increasing stack holding bar indices. Compute area when height decreases.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def largest_rectangle_area(heights):
    stack = []
    max_area = 0

```

```

heights.append(0)
for i, h in enumerate(heights):
    while stack and heights[stack[-1]] > h:
        height = heights[stack.pop()]
        width = i if not stack else i - stack[-1] - 1
        max_area = max(max_area, height * width)
    stack.append(i)
heights.pop()
return max_area

```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: All heights equal; strictly increasing or decreasing heights.

Sliding Window Maximum



Hard

Problem: Find max element in each sliding window of size k .

Approach: Monotonic Deque storing indices in decreasing order of values.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

from collections import deque

def max_sliding_window(nums, k):
    dq = deque()
    res = []
    for i, num in enumerate(nums):
        if dq and dq[0] < i - k + 1: dq.popleft()
        while dq and nums[dq[-1]] < num: dq.pop()
        dq.append(i)
        if i >= k - 1: res.append(nums[dq[0]])
    return res

```

Complexity: Time $O(n)$, Space $O(k)$.

Edge cases: $k = 1$; $k = \text{nums.length}$.



</> HackWithInfy 2026

Topic-wise practice bank

☰ 13/13 topics with full solutions

1. Arrays ✨ 2. Strings ✨ 3. Sliding Window ✨ 4. Binary Search ✨ 5. Hashing ✨ 6

7. Linked List

7 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

Reverse Linked List



Easy

Problem: Reverse a singly linked list.

Approach: Iterative three-pointer approach (prev, curr, nextTemp).

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def reverse_list(head):
    prev, curr = None, head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: Null head; single node list.

Linked List Cycle II



Easy

Problem: Find the node where the cycle begins in a linked list.

Approach: Floyd's Cycle Finding algorithm (Slow & Fast pointers). Reset slow to head upon intersection.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def detect_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
```

```
        if slow == fast:
            entry = head
            while entry != slow:
                entry, slow = entry.next, slow.next
            return entry
    return None
```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: No cycle; cycle at node 0 (head).

▼ Merge k Sorted Lists



Medium

Problem: Merge k sorted linked lists into one sorted list.

Approach: Min-Heap / PriorityQueue storing head of each list.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
import heapq

def merge_k_lists(lists):
    heap = []
    for i, l in enumerate(lists):
        if l: heapq.heappush(heap, (l.val, i, l))
    dummy = curr = ListNode(0)
    while heap:
        val, i, node = heapq.heappop(heap)
        curr.next = node
        curr = curr.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

Complexity: Time $O(N \log k)$, Space $O(k)$.

Edge cases: Empty array of lists; lists containing nulls.

▼ Add Two Numbers



Medium

Problem: Add two non-empty linked lists representing non-negative integers in reverse order.

Approach: Iterate through lists digit by digit, maintaining carry.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def add_two_numbers(l1, l2):
    dummy = curr = ListNode(0)
    carry = 0
    while l1 or l2 or carry:
        val = carry
```

```

        if l1: val += l1.val; l1 = l1.next
        if l2: val += l2.val; l2 = l2.next
        carry = val // 10
        curr.next = ListNode(val % 10)
        curr = curr.next
    return dummy.next

```

Complexity: Time $O(\max(N,M))$, Space $O(\max(N,M))$.

Edge cases: Final carry producing an additional node.

Flatten a Multilevel Doubly Linked List



Medium

Problem: Flatten a multilevel doubly linked list so nodes form a single-level doubly linked list.

Approach: Use a Stack to store next pointers when child pointers are encountered.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def flatten(head):
    if not head: return head
    curr = head
    while curr:
        if curr.child:
            nxt = curr.next
            child = curr.child
            curr.next = child; child.prev = curr; curr.child = None
            tail = child
            while tail.next: tail = tail.next
            tail.next = nxt
            if nxt: nxt.prev = tail
        curr = curr.next
    return head

```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: Deeply nested children; child at the end of list.

LRU Cache



Hard

Problem: Design LRU (Least Recently Used) Cache with get and put in $O(1)$.

Approach: HashMap (key $\rightarrow$ Node) + Doubly Linked List for ordering access.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

class Node:
    def __init__(self, k, v): self.key, self.val = k, v; self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity: int):

```

```

self.cap = capacity; self.map = {}
self.head, self.tail = Node(0, 0), Node(0, 0)
self.head.next = self.tail; self.tail.prev = self.head
def get(self, key: int) -> int:
    if key not in self.map: return -1
    n = self.map[key]; self._remove(n); self._insert(n)
    return n.val
def put(self, key: int, value: int) -> None:
    if key in self.map: self._remove(self.map[key])
    if len(self.map) == self.cap:
        del self.map[self.tail.prev.key]; self._remove(self.tail.prev)
    n = Node(key, value); self._insert(n); self.map[key] = n
def _remove(self, n): n.prev.next = n.next; n.next.prev = n.prev
def _insert(self, n): n.next = self.head.next; n.prev = self.head; self.head.next.prev = n; s

```

Complexity: Time $O(1)$, Space $O(\text{capacity})$.

Edge cases: Updating existing key; capacity 1.

Reverse Nodes in k-Group



Hard

Problem: Reverse nodes of a linked list k at a time.

Approach: Check if k nodes exist. If so, reverse the k sublist and recurse for the remainder.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def reverse_k_group(head, k):
    curr = head
    count = 0
    while curr and count != k:
        curr = curr.next
        count += 1
    if count == k:
        curr = reverse_k_group(curr, k)
        while count > 0:
            tmp = head.next
            head.next = curr
            curr = head
            head = tmp
            count -= 1
        head = curr
    return head

```

Complexity: Time $O(n)$, Space $O(n / k)$ call stack.

Edge cases: Length of list not multiple of k (leaves last part intact).



</> HackWithInfy 2026

Topic-wise practice bank

☰ 13/13 topics with full solutions

1. Arrays ✨ 2. Strings ✨ 3. Sliding Window ✨ 4. Binary Search ✨ 5. Hashing ✨ 6

8. Trees

8 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

Binary Tree Level Order Traversal



Easy

Problem: Return level order traversal of binary tree nodes' values.

Approach: BFS using a Queue. Process queue level by level.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
from collections import deque

def level_order(root):
    if not root: return []
    res, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res
```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: Empty tree (null root).

Balanced Binary Tree



Easy

Problem: Check if height of left & right subtrees of every node differs by ≤ 1 .

Approach: Bottom-up DFS returning height if balanced or -1 if unbalanced.

Java

Python

[Solve on LeetCode](#)

PYTHON


```
def is_balanced(root) -> bool:
    def check(node):
        if not node: return 0
        l = check(node.left)
        if l == -1: return -1
        r = check(node.right)
        if r == -1: return -1
        if abs(l - r) > 1: return -1
        return max(l, r) + 1
    return check(root) != -1
```

Complexity: Time $O(n)$, Space $O(h)$.

Edge cases: Single node tree; skewed tree.

✓ Lowest Common Ancestor of a Binary Tree



Medium

Problem: Find LCA of two given nodes p and q in binary tree.

Approach: Recursive DFS: return root if root == p || root == q. If both left & right return non-null, root is LCA.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def lowest_common_ancestor(root, p, q):
    if not root or root == p or root == q: return root
    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)
    if left and right: return root
    return left or right
```

Complexity: Time $O(n)$, Space $O(h)$.

Edge cases: p is ancestor of q.

✓ Diameter of Binary Tree



Medium

Problem: Find length of longest path between any two nodes in a tree.

Approach: DFS tracking global max of (leftHeight + rightHeight).

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def diameter_of_binary_tree(root) -> int:
    max_d = 0
    def height(node):
        nonlocal max_d
        if not node: return 0
        l, r = height(node.left), height(node.right)
        max_d = max(max_d, l + r)
```

```
        return max(l, r) + 1
    height(root)
    return max_d
```

Complexity: Time $O(n)$, Space $O(h)$.

Edge cases: Path does not pass through root.

Serialize and Deserialize Binary Tree



Medium

Problem: Design an algorithm to serialize and deserialize a binary tree.

Approach: Preorder traversal with null markers (e.g. 'null,').

Java

Python

[Solve on LeetCode](#)

PYTHON

```
class Codec:
    def serialize(self, root):
        if not root: return "N"
        return f"{root.val},{self.serialize(root.left)},{self.serialize(root.right)}"

    def deserialize(self, data):
        vals = iter(data.split(','))
        def build():
            val = next(vals)
            if val == 'N': return None
            node = TreeNode(int(val))
            node.left = build()
            node.right = build()
            return node
        return build()
```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: Negative values in tree; single node.

Construct Binary Tree from Preorder and Inorder Traversal



Medium

Problem: Reconstruct binary tree given preorder & inorder arrays.

Approach: Preorder gives root. Find root index in inorder array using HashMap to divide left & right subtrees.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def build_tree(preorder, inorder):
    in_map = {val: i for i, val in enumerate(inorder)}
    pre_idx = 0
    def helper(left, right):
        nonlocal pre_idx
        if left > right: return None
```

```

        val = preorder[pre_idx]
        pre_idx += 1
        root = TreeNode(val)
        idx = in_map[val]
        root.left = helper(left, idx - 1)
        root.right = helper(right, idx + 1, right)
        return root
    return helper(0, len(inorder) - 1)

```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: Single element tree.

Binary Tree Maximum Path Sum



Hard

Problem: Find maximum path sum where path goes from any node to any node.

Approach: DFS calculating max gain from subtrees (ignore negative gains with $\text{Math.max}(0, \text{gain})$).

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def max_path_sum(root) -> int:
    max_sum = float('-inf')
    def gain(node):
        nonlocal max_sum
        if not node: return 0
        l = max(0, gain(node.left))
        r = max(0, gain(node.right))
        max_sum = max(max_sum, node.val + l + r)
        return node.val + max(l, r)
    gain(root)
    return max_sum

```

Complexity: Time $O(n)$, Space $O(h)$.

Edge cases: All negative node values in tree.

Vertical Order Traversal of a Binary Tree



Hard

Problem: Return vertical order traversal of a binary tree.

Approach: BFS/DFS tracking (col, row, val). Sort by col, then row, then val.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

from collections import defaultdict, deque

def vertical_traversal(root):
    nodes = []
    q = deque([(root, 0, 0)])
    while q:

```

```
node, row, col = q.popleft()
if node:
    nodes.append((col, row, node.val))
    q.append((node.left, row + 1, col - 1))
    q.append((node.right, row + 1, col + 1))
nodes.sort()
res = {}
for col, row, val in nodes:
    res.setdefault(col, []).append(val)
return list(res.values())
```

Complexity: Time $O(n \log n)$, Space $O(n)$.

Edge cases: Multiple nodes at identical (col, row).



</> HackWithInfy 2026

Topic-wise practice bank

☰ 13/13 topics with full solutions

1. Arrays ✨ 2. Strings ✨ 3. Sliding Window ✨ 4. Binary Search ✨ 5. Hashing ✨ 6

9. Graphs

8 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

✓ Number of Connected Components in an Undirected Graph



Easy

Problem: Find number of connected components in n nodes with undirected edges.

Approach: Union-Find (Disjoint Set Union) with path compression.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def count_components(n, edges):
    parent = list(range(n))
    def find(i):
        if parent[i] == i: return i
        parent[i] = find(parent[i])
        return parent[i]
    comp = n
    for u, v in edges:
        r1, r2 = find(u), find(v)
        if r1 != r2:
            parent[r1] = r2
            comp -= 1
    return comp
```

Complexity: Time $O(E * \alpha(V))$, Space $O(V)$.

Edge cases: Graph with 0 edges (n components).

✓ Course Schedule



Medium

Problem: Determine if you can finish all courses given prerequisite dependencies.

Approach: Topological Sort via Kahn's Algorithm (In-degree array + BFS Queue).

Java

Python

[Solve on LeetCode](#)

PYTHON

```

from collections import deque, defaultdict

def can_finish(num_courses, prerequisites):
    in_degree = [0] * num_courses
    adj = defaultdict(list)
    for dest, src in prerequisites:
        adj[src].append(dest)
        in_degree[dest] += 1
    q = deque([i for i in range(num_courses) if in_degree[i] == 0])
    visited = 0
    while q:
        curr = q.popleft()
        visited += 1
        for nxt in adj[curr]:
            in_degree[nxt] -= 1
            if in_degree[nxt] == 0: q.append(nxt)
    return visited == num_courses

```

Complexity: Time $O(V + E)$, Space $O(V + E)$.

Edge cases: Cyclic dependencies present (e.g. $0 \rightarrow 1$ and $1 \rightarrow 0$).

Number of Islands



Medium

Problem: Count number of islands surrounded by water in a 2D binary grid.

Approach: DFS/BFS grid traversal. Sink visited land nodes from '1' to '0'.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def num_islands(grid):
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    def dfs(r, c):
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] == '0': return
        grid[r][c] = '0'
        dfs(r+1, c); dfs(r-1, c); dfs(r, c+1); dfs(r, c-1)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)
    return count

```

Complexity: Time $O(R * C)$, Space $O(R * C)$.

Edge cases: Grid containing only water '0'.

Network Delay Time (Dijkstra)



Medium

Problem: Find minimum time for signal to reach all n nodes from node k.

Approach: Dijkstra's Algorithm using a Min-Heap (dist, node).

Java

Python

[Solve on LeetCode](#)

PYTHON

```
import heapq, collections

def network_delay_time(times, n, k):
    adj = collections.defaultdict(list)
    for u, v, w in times: adj[u].append((v, w))
    pq = [(0, k)]
    dist = {}
    while pq:
        d, node = heapq.heappop(pq)
        if node not in dist:
            dist[node] = d
            for nxt, w in adj[node]:
                if nxt not in dist: heapq.heappush(pq, (d + w, nxt))
    return max(dist.values()) if len(dist) == n else -1
```

Complexity: Time $O(E \log V)$, Space $O(V + E)$.

Edge cases: Disconnected graph nodes.

✓ Clone Graph



Medium

Problem: Return a deep copy of a connected undirected graph.

Approach: HashMap mapping original Node → cloned Node + DFS traversal.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def clone_graph(node):
    old_to_new = {}
    def dfs(n):
        if not n: return None
        if n in old_to_new: return old_to_new[n]
        copy = Node(n.val)
        old_to_new[n] = copy
        for nei in n.neighbors:
            copy.neighbors.append(dfs(nei))
        return copy
    return dfs(node)
```

Complexity: Time $O(V + E)$, Space $O(V)$.

Edge cases: Single node with no neighbors.

✓ Word Ladder



Hard

Problem: Find length of shortest transformation sequence from beginWord to endWord.

Approach: BFS level-by-level state search. Replace each character 'a'-'z'.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
from collections import deque

def ladder_length(begin_word, end_word, word_list):
    words = set(word_list)
    if end_word not in words: return 0
    q = deque([(begin_word, 1)])
    while q:
        word, step = q.popleft()
        if word == end_word: return step
        for i in range(len(word)):
            for c in 'abcdefghijklmnopqrstuvwxyz':
                nxt = word[:i] + c + word[i+1:]
                if nxt in words:
                    words.remove(nxt)
                    q.append((nxt, step + 1))
    return 0
```

Complexity: Time $O(N * L * 26)$, Space $O(N)$.

Edge cases: endWord not in wordList.

▼ Alien Dictionary



Hard

Problem: Derive character ordering from sorted words list in alien language.

Approach: Build directed graph by comparing adjacent words, then perform Topological Sort.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
from collections import defaultdict, deque

def alien_order(words):
    adj = {c: set() for w in words for c in w}
    in_degree = {c: 0 for c in words for c in words}
    for i in range(len(words) - 1):
        w1, w2 = words[i], words[i+1]
        if len(w1) > len(w2) and w1.startswith(w2): return ""
        for c1, c2 in zip(w1, w2):
            if c1 != c2:
                if c2 not in adj[c1]:
                    adj[c1].add(c2)
                    in_degree[c2] += 1
                break
    q = deque([c for c in in_degree if in_degree[c] == 0])
    res = []
```



```

while q:
    c = q.popleft()
    res.append(c)
    for nxt in adj[c]:
        in_degree[nxt] -= 1
        if in_degree[nxt] == 0: q.append(nxt)
    return "".join(res) if len(res) == len(in_degree) else ""

```

Complexity: Time $O(C)$ total characters, Space $O(U)$ unique letters.

Edge cases: Invalid prefix order (e.g. 'abc' before 'ab'); cycle in ordering.

Cheapest Flights Within K Stops



Hard

Problem: Find cheapest price from src to dst with at most k stops.

Approach: Bellman-Ford algorithm executed for $k + 1$ iterations.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def find_cheapest_price(n, flights, src, dst, k):
    dist = [float('inf')] * n
    dist[src] = 0
    for _ in range(k + 1):
        tmp = list(dist)
        for u, v, w in flights:
            if dist[u] != float('inf') and dist[u] + w < tmp[v]:
                tmp[v] = dist[u] + w
        dist = tmp
    return dist[dst] if dist[dst] != float('inf') else -1

```

Complexity: Time $O(K * E)$, Space $O(V)$.

Edge cases: Destination unreachable within k stops.

10. Dynamic Programming

10 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

▼ Climbing Stairs / House Robber



Easy

Problem: Determine max money you can rob without robbing adjacent houses.

Approach: DP with 2 variables: rob1 (max up to i-2) and rob2 (max up to i-1).

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def rob(nums):
    rob1 = rob2 = 0
    for n in nums:
        rob1, rob2 = rob2, max(n + rob1, rob2)
    return rob2
```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: Empty array; single house.

▼ Coin Change



Easy

Problem: Fewest coins needed to make up given amount.

Approach: Bottom-up 1D DP table where $dp[i]$ = min coins for amount i.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def coin_change(coins, amount):
    dp = [amount + 1] * (amount + 1)
    dp[0] = 0
    for i in range(1, amount + 1):
        for coin in coins:
            if i >= coin:
```

```
dp[i] = min(dp[i], dp[i - coin] + 1)
return dp[amount] if dp[amount] <= amount else -1
```

Complexity: Time $O(\text{amount} * n)$, Space $O(\text{amount})$.

Edge cases: amount = 0; impossible to make change.

✓ Longest Increasing Subsequence



Medium

Problem: Find length of longest strictly increasing subsequence.

Approach: Patience Sorting / Binary Search on tail values in $O(n \log n)$.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
import bisect

def length_of_lis(nums):
    tails = []
    for x in nums:
        idx = bisect.bisect_left(tails, x)
        if idx == len(tails): tails.append(x)
        else: tails[idx] = x
    return len(tails)
```

Complexity: Time $O(n \log n)$, Space $O(n)$.

Edge cases: Strictly decreasing array.

✓ Partition Equal Subset Sum



Medium

Problem: Check if array can be partitioned into two subsets with equal sum.

Approach: 0/1 Knapsack DP using boolean array for target sum = totalSum / 2.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def can_partition(nums):
    total = sum(nums)
    if total % 2 != 0: return False
    target = total // 2
    dp = {0}
    for n in nums:
        dp |= {x + n for x in dp if x + n <= target}
    return target in dp
```

Complexity: Time $O(N * \text{Sum})$, Space $O(\text{Sum})$.

Edge cases: Odd total sum.

▼ Edit Distance

Medium

Problem: Find min operations (insert, delete, replace) to convert word1 to word2.

Approach: 2D DP grid: $dp[i][j]$ = min edits to convert word1[0..i] to word2[0..j].

JavaPython[Solve on LeetCode](#)

PYTHON

```
def min_distance(word1: str, word2: str) -> int:
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i-1] == word2[j-1]: dp[i][j] = dp[i-1][j-1]
            else: dp[i][j] = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

Complexity: Time $O(M * N)$, Space $O(M * N)$.

Edge cases: Empty string.

▼ Unique Paths II

Medium

Problem: Find unique paths from top-left to bottom-right in grid with obstacles.

Approach: 2D DP table. Set $dp[r][c] = 0$ if obstacle, else $dp[r-1][c] + dp[r][c-1]$.

JavaPython[Solve on LeetCode](#)

PYTHON

```
def unique_paths_with_obstacles(obstacle_grid):
    w = len(obstacle_grid[0])
    dp = [0] * w
    dp[0] = 1
    for row in obstacle_grid:
        for j in range(w):
            if row[j] == 1: dp[j] = 0
            elif j > 0: dp[j] += dp[j - 1]
    return dp[-1]
```

Complexity: Time $O(R * C)$, Space $O(C)$.

Edge cases: Obstacle at start or end.

▼ Matrix Chain Multiplication

Hard

Problem: Find minimum cost to multiply a chain of matrices.

Approach: Interval DP over chain length subproblems.

[Java](#)[Python](#)[Solve on LeetCode](#)

PYTHON

```
def matrix_multiplication(arr):
    n = len(arr)
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n):
        for i in range(1, n - length + 1):
            j = i + length - 1
            dp[i][j] = float('inf')
            for k in range(i, j):
                cost = dp[i][k] + dp[k+1][j] + arr[i-1] * arr[k] * arr[j]
                dp[i][j] = min(dp[i][j], cost)
    return dp[1][n-1]
```

Complexity: Time $O(n^3)$, Space $O(n^2)$.

Edge cases: Minimum chain length of 3.

▼ Palindrome Partitioning II

**Hard**

Problem: Minimum cuts needed for a palindrome partitioning of string s.

Approach: Expand around center to find palindromes, updating 1D min-cut array.

[Java](#)[Python](#)[Solve on LeetCode](#)

PYTHON

```
def min_cut(s: str) -> int:
    n = len(s)
    cuts = list(range(n))
    def expand(l, r):
        while l >= 0 and r < n and s[l] == s[r]:
            new_cut = 0 if l == 0 else cuts[l - 1] + 1
            cuts[r] = min(cuts[r], new_cut)
            l -= 1; r += 1
    for i in range(n):
        expand(i, i)
        expand(i, i + 1)
    return cuts[-1]
```

Complexity: Time $O(n^2)$, Space $O(n)$.

Edge cases: Entire string is already a palindrome (0 cuts).

▼ Burst Balloons

**Hard**

Problem: Find maximum coins collected by bursting balloons strategically.

Approach: Interval DP: consider which balloon is burst LAST in interval $[i, j]$.

[Java](#)[Python](#)[Solve on LeetCode](#)



```
def max_balloons(nums):  
    val = [1] + nums + [1]  
    n = len(val)  
    dp = [[0] * n for _ in range(n)]  
    for length in range(2, n):  
        for left in range(n - length):  
            right = left + length  
            for i in range(left + 1, right):  
                dp[left][right] = max(dp[left][right],  
                                       val[left] * val[i] * val[right] + dp[left][i] + dp[i][right])  
    return dp[0][n-1]
```

Complexity: Time $O(n^3)$, Space $O(n^2)$.

Edge cases: Single balloon.

▼ Wildcard Matching



Hard

Problem: Implement wildcard pattern matching with '?' and '*'.

Approach: Greedy 2-pointer approach tracking last star index and matched position.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def is_match(s: str, p: str) -> bool:  
    s_idx = p_idx = match = 0  
    star_idx = -1  
    while s_idx < len(s):  
        if p_idx < len(p) and (p[p_idx] in ('?', s[s_idx])):  
            s_idx += 1; p_idx += 1  
        elif p_idx < len(p) and p[p_idx] == '*':  
            star_idx = p_idx; match = s_idx; p_idx += 1  
        elif star_idx != -1:  
            p_idx = star_idx + 1; match += 1; s_idx = match  
        else: return False  
    return all(c == '*' for c in p[p_idx:])
```

Complexity: Time $O(S * P)$ worst case, $O(S)$ average, Space $O(1)$.

Edge cases: Pattern composed entirely of '*'.



HackWithInfy 2026

Topic-wise practice bank

13/13 topics with full solutions

1. Arrays 2. Strings 3. Sliding Window 4. Binary Search 5. Hashing 6

11. Greedy

7 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

Non-overlapping Intervals



Easy

Problem: Find minimum intervals to remove to make remaining intervals non-overlapping.

Approach: Greedy sort by end time. Keep track of previous end time.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def erase_overlap_intervals(intervals):
    intervals.sort(key=lambda x: x[1])
    removals = 0
    prev_end = intervals[0][1]
    for start, end in intervals[1:]:
        if start < prev_end: removals += 1
        else: prev_end = end
    return removals
```

Complexity: Time $O(n \log n)$, Space $O(1)$.

Edge cases: Intervals touching at endpoint (e.g. [1,2] and [2,3] do not overlap).

Jump Game II



Easy

Problem: Find minimum jumps required to reach the last index.

Approach: Greedy BFS: track current jump boundary and max reachable index within boundary.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def jump(nums):
    jumps = curr_end = max_reach = 0
    for i in range(len(nums) - 1):
        max_reach = max(max_reach, i + nums[i])
        if i == curr_end:
```

```
jumps += 1
curr_end = max_reach
return jumps
```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: Single element array (0 jumps required).

▼ Gas Station



Medium

Problem: Find starting gas station index to travel circuit clockwise once.

Approach: If total gas < total cost, return -1. Otherwise, reset start index whenever tank drops below 0.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def can_complete_circuit(gas, cost):
    if sum(gas) < sum(cost): return -1
    tank = start = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:
            start = i + 1
            tank = 0
    return start
```

Complexity: Time $O(n)$, Space $O(1)$.

Edge cases: Circuit starting at last index.

▼ Task Scheduler



Medium

Problem: Find minimum CPU intervals needed to complete tasks given cooldown n.

Approach: Math/Greedy formula based on max frequency task count.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
from collections import Counter

def least_interval(tasks, n):
    counts = Counter(tasks)
    max_freq = max(counts.values())
    max_count = sum(1 for v in counts.values() if v == max_freq)
    return max(len(tasks), (max_freq - 1) * (n + 1) + max_count)
```

Complexity: Time $O(N)$, Space $O(1)$.

Edge cases: $n = 0$ (no cooldown required).

Minimum Number of Arrows to Burst Balloons



Medium

Problem: Find minimum arrows needed to burst all balloons.

Approach: Greedy sort by end coordinate; shoot arrow at balloon's end coordinate.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def find_min_arrow_shots(points):
    points.sort(key=lambda x: x[1])
    arrows = 1
    pos = points[0][1]
    for start, end in points[1:]:
        if start > pos:
            arrows += 1
            pos = end
    return arrows
```

Complexity: Time $O(n \log n)$, Space $O(1)$.

Edge cases: Integer overflow on sorting coordinates (use Integer.compare).

Course Schedule III



Hard

Problem: Find maximum courses you can take given duration & deadline.

Approach: Greedy sort by deadline + Max-Heap for replacing long duration courses.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
import heapq

def schedule_course(courses):
    courses.sort(key=lambda x: x[1])
    heap = []
    time = 0
    for dur, end in courses:
        if time + dur <= end:
            time += dur
            heapq.heappush(heap, -dur)
        elif heap and -heap[0] > dur:
            time += dur + heapq.heappop(heap)
            heapq.heappush(heap, -dur)
    return len(heap)
```

Complexity: Time $O(n \log n)$, Space $O(n)$.

Edge cases: Duration exceeds deadline.

Problem: Distribute minimum candies to children satisfying rating constraints.

Approach: Two passes: left-to-right pass then right-to-left pass.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```
def candy(ratings):
    n = len(ratings)
    candies = [1] * n
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]: candies[i] = candies[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]: candies[i] = max(candies[i], candies[i + 1] + 1)
    return sum(candies)
```

Complexity: Time $O(n)$, Space $O(n)$.

Edge cases: Equal ratings.



HackWithInfy 2026

Topic-wise practice bank

13/13 topics with full solutions

1. Arrays
2. Strings
3. Sliding Window
4. Binary Search
5. Hashing
- 6

12. Backtracking

7 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

Permutations / Subsets



Easy

Problem: Return all possible subsets (the power set) of unique elements.

Approach: Backtracking DFS: decide whether to include element at current index.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def subsets(nums):
    res = []
    def backtrack(start, curr):
        res.append(list(curr))
        for i in range(start, len(nums)):
            curr.append(nums[i])
            backtrack(i + 1, curr)
            curr.pop()
    backtrack(0, [])
    return res
```

Complexity: Time $O(2^n)$, Space $O(n)$.

Edge cases: Empty set.

Combination Sum



Medium

Problem: Find unique combinations summing to target (elements reusable).

Approach: Backtracking passing start index to allow element reuse.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def combination_sum(candidates, target):
    res = []
    def backtrack(remain, curr, start):
```

```

    if remain == 0: res.append(list(curr)); return
    if remain < 0: return
    for i in range(start, len(candidates)):
        curr.append(candidates[i])
        backtrack(remain - candidates[i], curr, i)
        curr.pop()
    backtrack(target, [], 0)
    return res

```

Complexity: Time $O(2^{\text{target}})$, Space $O(\text{target})$.

Edge cases: Target smaller than smallest candidate.

N-Queens



Medium

Problem: Place N queens on N x N board such that no two queens attack each other.

Approach: Backtrack row by row tracking occupied columns and diagonals.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def solve_n_queens(n):
    res = []
    cols, d1, d2 = set(), set(), set()
    board = [ "." * n for _ in range(n) ]
    def backtrack(r):
        if r == n:
            res.append("".join(row) for row in board)
            return
        for c in range(n):
            if c not in cols and (r + c) not in d1 and (r - c) not in d2:
                cols.add(c); d1.add(r + c); d2.add(r - c)
                board[r][c] = "Q"
                backtrack(r + 1)
                cols.remove(c); d1.remove(r + c); d2.remove(r - c)
                board[r][c] = "."
    backtrack(0)
    return res

```

Complexity: Time $O(N!)$, Space $O(N^2)$.

Edge cases: N = 1.

Sudoku Solver



Medium

Problem: Solve a Sudoku puzzle by filling empty cells '.'.

Approach: Backtracking attempting digits '1'-'9' for empty cells.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

def solve_sudoku(board):
    def is_valid(r, c, ch):
        for i in range(9):
            if board[r][i] == ch or board[i][c] == ch or board[3*(r//3) + i//3][3*(c//3) + i%3] == ch:
                return False
        return True
    def solve():
        for r in range(9):
            for c in range(9):
                if board[r][c] == '.':
                    for ch in '123456789':
                        if is_valid(r, c, ch):
                            board[r][c] = ch
                            if solve(): return True
                            board[r][c] = '.'
                    return False
        return True
    solve()

```

Complexity: Time $O(9^9(N^2))$, Space $O(1)$.

Edge cases: Invalid puzzle state.

Word Search



Medium

Problem: Check if word exists in 2D board of characters.

Approach: Grid DFS with backtracking (temporarily overwrite visited cell).

Java

Python

[Solve on LeetCode](#)

PYTHON

```

def exist(board, word):
    rows, cols = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word): return True
        if r < 0 or c < 0 or r >= rows or c >= cols or board[r][c] != word[i]: return False
        temp = board[r][c]
        board[r][c] = '#'
        res = dfs(r+1, c, i+1) or dfs(r-1, c, i+1) or dfs(r, c+1, i+1) or dfs(r, c-1, i+1)
        board[r][c] = temp
        return res
    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0): return True
    return False

```

Complexity: Time $O(R * C * 4^L)$, Space $O(L)$.

Edge cases: Word length exceeds board total cell count.

▼ Palindrome Partitioning

Hard

Problem: Partition string s such that every substring is a palindrome.

Approach: Backtrack through indices, checking palindrome property before recursing.

JavaPython[Solve on LeetCode](#)

PYTHON

```
def partition(s):
    res = []
    def backtrack(start, curr):
        if start == len(s): res.append(list(curr)); return
        for end in range(start + 1, len(s) + 1):
            sub = s[start:end]
            if sub == sub[::-1]:
                curr.append(sub)
                backtrack(end, curr)
                curr.pop()
    backtrack(0, [])
    return res
```

Complexity: Time $O(N * 2^N)$, Space $O(N)$.

Edge cases: Single character string.

▼ Restore IP Addresses

Hard

Problem: Form all valid IP addresses from digit string s .

Approach: Backtracking forming 4 segments (each between 0 and 255 with no leading zeros).

JavaPython[Solve on LeetCode](#)

PYTHON

```
def restore_ip_addresses(s):
    res = []
    def backtrack(start, curr):
        if len(curr) == 4:
            if start == len(s): res.append(".".join(curr))
            return
        for length in range(1, 4):
            if start + length <= len(s):
                part = s[start:start+length]
                if (part.startswith('0') and len(part) > 1) or int(part) > 255: continue
                curr.append(part)
                backtrack(start + length, curr)
                curr.pop()
    backtrack(0, [])
    return res
```

Complexity: Time $O(3^4) = O(1)$, Space $O(1)$.

Edge cases: String length < 4 or > 12.



HackWithInfy 2026

Topic-wise practice bank

13/13 topics with full solutions

1. Arrays 2. Strings 3. Sliding Window 4. Binary Search 5. Hashing 6

13. Advanced Data Structures

6 problems

Ordered Easy to Hard — mirrors the real 3-question round difficulty curve.

Word Search II (Trie)



Medium

Problem: Find all words from dictionary present on 2D board.

Approach: Build Trie from word list + Grid DFS traversal matching Trie nodes.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
class TrieNode:
    def __init__(self): self.children = {}; self.word = None

def find_words(board, words):
    root = TrieNode()
    for w in words:
        curr = root
        for c in w:
            curr = curr.children.setdefault(c, TrieNode())
            curr.word = w
    rows, cols = len(board), len(board[0])
    res = []
    def dfs(r, c, parent):
        ch = board[r][c]
        curr = parent.children[ch]
        if curr.word:
            res.append(curr.word); curr.word = None
        board[r][c] = '#'
        for dr, dc in [(1,0),(-1,0),(0,1),(0,-1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] in curr.children:
                dfs(nr, nc, curr)
        board[r][c] = ch
    for r in range(rows):
        for c in range(cols):
            if board[r][c] in root.children: dfs(r, c, root)
    return res
```


Complexity: Time $O(R * C * 4^L)$, Space $O(\text{Sum of Word Lengths})$.

Edge cases: Duplicate words in board; prefix shared across dictionary.

▼ Number of Provinces (Union-Find)



Medium

Problem: Find number of connected provinces in adjacency matrix.

Approach: Union-Find with path compression.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
def find_circle_num(is_connected):
    n = len(is_connected)
    parent = list(range(n))
    def find(i):
        if parent[i] == i: return i
        parent[i] = find(parent[i])
        return parent[i]
    count = n
    for i in range(n):
        for j in range(i + 1, n):
            if is_connected[i][j] == 1:
                r1, r2 = find(i), find(j)
                if r1 != r2:
                    parent[r1] = r2
                    count -= 1
    return count
```

Complexity: Time $O(N^2 * \alpha(N))$, Space $O(N)$.

Edge cases: Entire matrix is 0s (n separate provinces).

▼ Range Sum Query - Mutable (Segment Tree / BIT)



Medium

Problem: Update single array elements and query range sums in $O(\log n)$.

Approach: Binary Indexed Tree (Fenwick Tree) / Segment Tree.

Java

Python

[Solve on LeetCode](#)

PYTHON

```
class NumArray:
    def __init__(self, nums):
        self.n = len(nums)
        self.nums = list(nums)
        self.tree = [0] * (self.n + 1)
        for i, val in enumerate(nums):
            self._add(i + 1, val)

    def _add(self, i, val):
        while i <= self.n:
```

```

        self.tree[i] += val
        i += i & -i

def update(self, index: int, val: int) -> None:
    diff = val - self.nums[index]
    self.nums[index] = val
    self._add(index + 1, diff)

def _query(self, i):
    s = 0
    while i > 0:
        s += self.tree[i]
        i -= i & -i
    return s

def sumRange(self, left: int, right: int) -> int:
    return self._query(right + 1) - self._query(left)

```

Complexity: Time $O(\log n)$ update & sumRange, Space $O(n)$.

Edge cases: Single element array.

▼ All O`one Data Structure



Hard

Problem: Design data structure supporting inc(key), dec(key), getMaxKey(), getMinKey() in $O(1)$.

Approach: Doubly Linked List of Bucket nodes (storing count & set of keys) + HashMap.

Java

Python

[Solve on LeetCode](#)

PYTHON

```

class Bucket:
    def __init__(self, count):
        self.count = count; self.keys = set()
        self.prev = self.next = None

class AllOne:
    def __init__(self):
        self.head, self.tail = Bucket(0), Bucket(0)
        self.head.next = self.tail; self.tail.prev = self.head
        self.map = {}

    def inc(self, key: str) -> None:
        if key in self.map:
            b = self.map[key]; b.keys.remove(key)
            nxt = b.next
            if nxt == self.tail or nxt.count != b.count + 1:
                nxt = self._add_after(b, Bucket(b.count + 1))
            nxt.keys.add(key); self.map[key] = nxt
            if not b.keys: self._remove(b)
        else:
            first = self.head.next
            if first == self.tail or first.count != 1:

```

```

        first = self._add_after(self.head, Bucket(1))
        first.keys.add(key); self.map[key] = first

def dec(self, key: str) -> None:
    b = self.map[key]; b.keys.remove(key)
    if b.count == 1: del self.map[key]
    else:
        prv = b.prev
        if prv == self.head or prv.count != b.count - 1:
            prv = self._add_after(b.prev, Bucket(b.count - 1))
        prv.keys.add(key); self.map[key] = prv
    if not b.keys: self._remove(b)

def getMaxKey(self) -> str: return "" if self.tail.prev == self.head else next(iter(self.tail
def getMinKey(self) -> str: return "" if self.head.next == self.tail else next(iter(self.head
def _add_after(self, p, n): n.next = p.next; n.prev = p; p.next.prev = n; p.next = n; return
def _remove(self, b): b.prev.next = b.next; b.next.prev = b.prev

```

Complexity: Time $O(1)$, Space $O(N)$.

Edge cases: Empty data structure returns empty string.

Find Median from Data Stream



Hard

Problem: Continuously compute median from a stream of numbers.

Approach: Two Heaps: Max-Heap for lower half, Min-Heap for upper half.

Java

Python

[Solve on LeetCode](#) 

PYTHON

```

import heapq

class MedianFinder:
    def __init__(self):
        self.small = [] # Max-heap
        self.large = [] # Min-heap
    def addNum(self, num: int) -> None:
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.small) < len(self.large):
            heapq.heappush(self.small, -heapq.heappop(self.large))
    def findMedian(self) -> float:
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2.0

```

Complexity: Time $O(\log n)$ addNum, $O(1)$ findMedian, Space $O(n)$.

Edge cases: Odd vs Even total count of elements.

▼ The Skyline Problem

Hard

Problem: Compute skyline formed by collective 3D building outlines.

Approach: Sweep-Line Algorithm: Process building x-boundaries with a Max-Heap / TreeMap.

JavaPython[Solve on LeetCode](#)

PYTHON

```
import heapq

def get_skyline(buildings):
    events = []
    for L, R, H in buildings:
        events.append((L, -H, R))
        events.append((R, 0, 0))
    events.sort()
    res = [[0, 0]]
    live = [(0, float('inf'))] # (-height, right)
    for x, negH, R in events:
        while live[0][1] <= x:
            heapq.heappop(live)
        if negH < 0:
            heapq.heappush(live, (negH, R))
        if res[-1][1] != -live[0][0]:
            res.append([x, -live[0][0]])
    return res[1:]
```

Complexity: Time $O(n \log n)$, Space $O(n)$.

Edge cases: Overlapping building edges; identical height buildings.